UNIVERSIDADE FEDERAL DO CEARÁ (UFC) • ENGENHARIA / CIÊNCIA DA COMPUTAÇÃO

Trabalho 1: Aprendizagem Federada com gRPC

Guia Técnico da Arquitetura, Manual de Execução para Colegas e Roteiro de Apresentação

1. Visão Geral e Fundamentos Teóricos

Este trabalho aborda a implementação distribuída de **Aprendizagem Federada (Federated Learning)** utilizando o framework **gRPC** e a linguagem Python. O modelo treinado é uma **Regressão Logística multiclasse** sobre o clássico dataset **Iris** (classificação de três espécies de flores a partir do comprimento e largura de pétalas e sépalas).

Conceito Central (Mover a computação aos dados): No aprendizado centralizado convencional, todos os dados precisam ser transferidos para um servidor central. Isso cria graves problemas de conformidade legal (ex.: LGPD e GDPR), violação de privacidade e alto consumo de largura de banda. Na Aprendizagem Federada, *os dados privados nunca saem dos nós clientes*. O que é transmitido na rede são unicamente os parâmetros estatísticos do modelo (pesos e bias) e métricas locais.

O Ciclo Federado em 5 Etapas:

1. **Inicialização e Distribuição:** O servidor mantém o modelo global e disponibiliza os parâmetros da rodada corrente aos clientes participantes.
2. **Treinamento Local:** Cada cliente recebe os pesos globais e treina seu modelo local com sua partição privada (80% treino local, 20% teste).
3. **Submissão de Atualizações:** Cada cliente envia seus pesos pós-treino e métricas (loss e acurácia) ao servidor.
4. **Agregação FedAvg (Federated Averaging):** Quando todos os clientes respondem, o servidor calcula a média ponderada dos parâmetros com base no número de exemplos locais de cada nó.
5. **Avaliação e Convergência:** O servidor avalia o modelo agregado sobre o conjunto de teste global e inicia a próxima rodada até atingir o total estipulado.

2. Arquitetura da Solução e Estrutura dos Arquivos

Arquivo	Papel e Responsabilidade no Sistema
federated.proto	Contrato de interface IDL. Define as mensagens Protobuf (ClientHello, ModelParameters, GlobalModel, ClientUpdate, UpdateAck) e o serviço gRPC FederatedLearning.
federated_pb2*.py	Código compilado gerado pelo protoc. Provê classes de serialização e stubs de comunicação cliente-servidor.
common_fl.py	Módulo de apoio. Particiona o Iris em splits IID com flwr-datasets, instancia o template Scikit-Learn, computa aggregate_fedavg e avalia o modelo global.
student_fl_client_grpc.py	Nó Cliente. Converte pesos NumPy/Protobuf, executa o laço de polling com should_wait, chama local_train e trata desconexões graciosamente.
student_fl_server_grpc.py	Nó Servidor. Implementa o Servicer gRPC thread-safe (via threading.Lock), valida updates de rodadas, agrega via FedAvg e avalia o modelo global.
tests/	Bateria com 15 testes unitários em 5 arquivos que validam passo a passo a correção da serialização, validação e agregação.

3. Manual Passo a Passo de Instalação e Execução

Siga este roteiro para executar o projeto do zero em qualquer computador com Python 3.9+ instalado:

Passo 3.1: Obter o Código e Navegar até a Pasta

```
git clone https://github.com/NicolasHarnisch/Trabalho-SD.git
cd Trabalho-SD/codigo
```

Passo 3.2: Criar e Ativar o Ambiente Virtual

```
# Opcao 1 (Recomendada via uv):
uv venv --python 3.14
.\.venv\Scripts\Activate.ps1    # No Windows (PowerShell)
source .venv/bin/activate         # No Linux / macOS

# Opcao 2 (Python padrao):
python -m venv .venv
.\.venv\Scripts\activate         # No Windows
source .venv/bin/activate         # No Linux / macOS
```

Passo 3.3: Instalar as Dependências

```
pip install grpcio grpcio-tools flwr-datasets scikit-learn numpy pandas
```

Passo 3.4: Executar a Bateria Completa de Testes Unitários

Entre na pasta start e rode o runner de testes:

```
cd start
python -m unittest discover -s tests -v
```

Resultado esperado: Ran 15 tests in ... OK.

Passo 3.5: Execução Prática Distribuída (Servidor e Múltiplos Clientes)

Abra terminais independentes na pasta codigo/start com o ambiente ativo (ou usando o executável do venv):

```
# [Terminal 1 - Servidor Coordenador]:
python student_fl_server_grpc.py --host 127.0.0.1 --port 50051 --num-clients 3 --rounds 5

# [Terminal 2 - Cliente 0]:
python student_fl_client_grpc.py --cid 0 --server-address 127.0.0.1:50051 --num-clients 3

# [Terminal 3 - Cliente 1]:
python student_fl_client_grpc.py --cid 1 --server-address 127.0.0.1:50051 --num-clients 3

# [Terminal 4 - Cliente 2]:
python student_fl_client_grpc.py --cid 2 --server-address 127.0.0.1:50051 --num-clients 3
```

Dica Windows: Se python chamar o interpretador global do sistema, utilize explicitamente ..\.venv\Scripts\python.exe no lugar de python.

4. Roteiro de Apresentação e Demonstração

Siga esta estrutura para uma apresentação clara, segura e profissional:

Etapa	O que dizer e demonstrar
1. Pitch Inicial (1 minuto)	Apresente o objetivo: *'Nosso projeto simula Aprendizagem Federada sobre o dataset Iris usando gRPC em Python. O foco central é a preservação da privacidade: os dados nunca saem dos clientes; apenas parâmetros treinados e métricas são transmitidos pela rede com Protocol Buffers.'*
2. Bateria de Testes (1 minuto)	Execute no terminal: python -m unittest discover -s tests -v. Explique que foram construídos 15 testes unitários cobrindo desde a conversão de tensores NumPy até regras de concorrência e agregação de parâmetros no servidor.
3. Execução ao Vivo (2 minutos)	Inicie o servidor e os 3 clientes em terminais lado a lado. Destaque: 1) O servidor esperando todos os clientes (1/3, 2/3, 3/3); 2) A agregação FedAvg ocorrendo no final de cada rodada; 3) A **convergência** do modelo (a perda diminui e a acurácia supera 95%); 4) O encerramento limpo com sinalização done=True.

5. Perguntas Frequentes do Professor (FAQ Técnico)

P1: Por que o FedAvg utiliza média ponderada e não uma média aritmética simples?

R: Porque em cenários federados reais os clientes podem ter volumes diferentes de dados. A ponderação pelo número de exemplos locais (peso = num_examples / total_examples) garante que cada amostra tenha a mesma influência estatística no modelo agregado final, evitando distorções causadas por clientes com pouquíssimas amostras.

P2: Como foi tratada a concorrência no servidor gRPC?

R: Como o servidor gRPC atende requisições de clientes em múltiplas threads através de um ThreadPoolExecutor, utilizamos uma trava de exclusão mútua (threading.Lock()) nos métodos SubmitUpdate e GetGlobalModel. Isso impede condições de corrida ao atualizar o dicionário de updates recebidos e avançar a variável current_round.

P3: Como as matrizes NumPy foram trafegadas pelo Protocol Buffers se não há tipo de matriz nativo?

R: Através do achatamento (*flattening*): a matriz 2D de coeficientes foi achatada em um vetor 1D de números float usando coef.ravel().tolist() e as dimensões originais foram guardadas na lista coef_shape. Na recepção, o método np.reshape() reconstrói a matriz exatamente com suas dimensões originais.

P4: O que acontece se um cliente tentar enviar um update de uma rodada desatualizada?

R: O método _validate_update no servidor verifica se request.round == self.current_round. Se o cliente estiver atrasado ou enviar um update duplicado na mesma rodada, o servidor rejeita imediatamente respondendo UpdateAck(accepted=False, message=...) sem poluir o buffer de agregação.